

CHAPTER 5

Knowledge Graph Learning

*A Plain-English Companion to
AI-Powered Search*

based on the work of
Trey Grainger · Doug Turnbull · Max Irwin

Generated 2026-05-12

About this book

This is a friendly retelling of Chapter 5 of *AI-Powered Search*. The original chapter is technical, careful, and full of the working code you'd need to build the systems it describes. That's exactly what its intended audience — search engineers — wants from it.

This version is written for a different reader. Someone who's curious about how modern search works, but doesn't write code for a living. Someone who can follow a diagram, understands what an analogy is asking them to do, and would rather read a paragraph than a code listing.

If you're a product manager, an analyst, a designer, an engineer in another part of the stack — this is for you. You'll come away from this chapter with a working mental model of how a search engine can be turned into a knowledge graph, why anyone would want to do that, and what kinds of problems that solves. You won't come away knowing how to write the code yourself, and that's fine.

What you'll find in this chapter

Five sections, each building on the last:

1. **Knowledge graphs: connecting the dots.** What a knowledge graph is, the two pieces every graph is built from, and the three ways people put them together.
1. **Your search engine already knows more than you think.** A surprising claim — the search engine you may already have is secretly a graph database — and how to see that for yourself.
1. **Teaching a computer to read between the lines.** Two ways to pull facts out of plain text and put them into a knowledge graph.
1. **Finding hidden connections in your words.** The heart of the chapter. Watch a search engine discover that *advil*, *motrin*, and *aleve* are related — without anyone telling it.
1. **Putting it all together: smarter search.** How the pieces fit, and a look at what they let you build.

You'll see a sprinkle of callouts throughout:

TIP

Practical advice you can use right away.

WARNING

Things that sound right but get people in trouble.

TECHNICAL

Skippable. For the curious; not needed to follow the argument.

REMEMBER

A one-line takeaway worth holding on to.

The chapter ends with a glossary of every term we'll introduce, and a one-page cheat sheet you can keep nearby.

Ready when you are.

Contents

	Title
Section 1	Knowledge Graphs: Connecting the Dots
Section 2	Your Search Engine Already Knows More Than You Think
Section 3	Teaching a Computer to Read Between the Lines
Section 4	Finding Hidden Connections in Your Words
Section 5	Putting It All Together: Smarter Search
Reference	Glossary · Cheat sheet

1. Knowledge Graphs: Connecting the Dots

If you've ever drawn a family tree, you've made a knowledge graph.

You drew little circles or boxes for each person. You drew lines between them to show how they're related — *is the parent of*, *is married to*, *is the sibling of*. You probably didn't think of yourself as building a graph database. But that's what you were doing. A graph is just dots connected by lines. A knowledge graph is the same idea, but the dots stand for real things in the world and the lines stand for real relationships between them.

You've seen these graphs in action many times even if you haven't named them.

When you search for a movie on Google and a panel appears on the right with the director, the cast, the year, the budget, the box office, the studio — that's coming from a knowledge graph. Each fact is a line between two dots.

When Netflix suggests "because you watched *X*, you might like *Y*," there's a graph in the background mapping movies to other movies, to actors, to themes, to viewers. A traversal through that graph is what produced the recommendation.

When LinkedIn says "people you may know," it's not magic. It's a graph of who knows whom, walked one or two steps out from your name.

The trick this chapter is about is a particular kind of knowledge graph: one that a search engine builds for itself, automatically, from the documents you already have. By the end of the chapter you'll see why that's a much bigger deal than it sounds.

But first, let's nail down what a graph actually is.

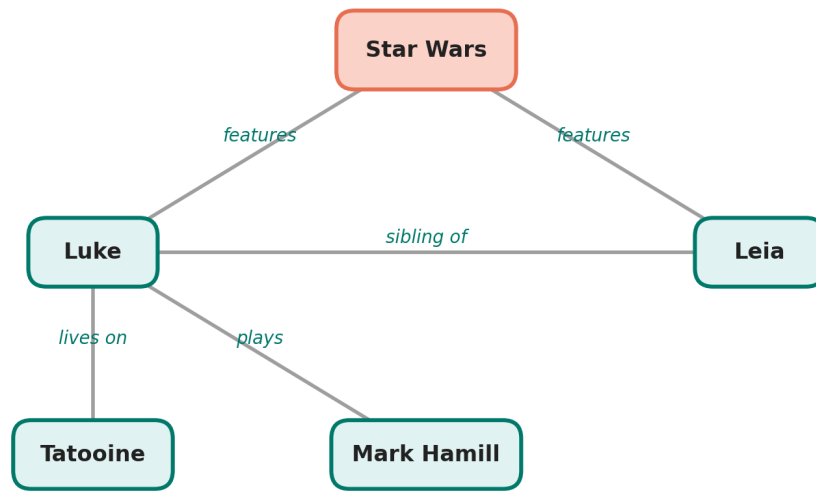
The two pieces of every graph

Every graph in the world is made of just two things.

Nodes. Sometimes called *vertices* (because mathematicians like to give simple things long names). A node is a *thing*. A person, a place, a movie, a word, a concept, a product, a paper, a recipe, a date — it doesn't matter what. If you can name it, it can be a node.

Edges. An edge is a *relationship between two things*. It connects two nodes and usually has a label that says how they're related. *is the parent of*, *contains*, *was directed by*, *was published in*.

That's all the vocabulary you need to start. Here's a small graph drawn with just those two pieces.



Read it like a sentence. Star Wars features Luke. Star Wars features Leia. Luke lives on Tatooine. Mark Hamill plays Luke. Luke is a sibling of Leia.

Five facts, five edges, five nodes. You could keep going forever. Add Han Solo, the Millennium Falcon, the Death Star, every movie in the franchise, every actor in every movie, the years they came out, the directors, the cinematographers. The graph just gets bigger. It doesn't get more complicated. It's still nodes and edges.

TIP

When you read about a knowledge graph that has "a billion edges" or "100 million nodes," don't be intimidated. It's the same Luke-and-Leia idea, scaled up. The hard problem isn't drawing the graph — it's deciding which nodes and edges should exist in the first place.

When a graph becomes a knowledge graph

Not every graph is a knowledge graph.

A road map is a graph. The intersections are the nodes, the streets are the edges. But we wouldn't usually call a road map a "knowledge graph" — its job is to help you navigate, not to capture facts about how the world fits together.

The wiring diagram of a microwave is a graph. The components are the nodes, the wires are the edges. Again, useful, but not a knowledge graph.

The shift is fuzzy, but in practice a graph becomes a *knowledge* graph when:

1. The nodes stand for real-world things people can name — people, products, ideas, places.
2. The edges describe meaningful relationships — *is a*, *wrote*, *married to*, *located in*, *contains*.
3. You can use the graph to answer questions you couldn't easily answer otherwise.

That third part is the punchline. The reason anyone bothers building one of these is to answer questions that flat-text search can't. Questions like:

Show me drugs that work the same way as advil.

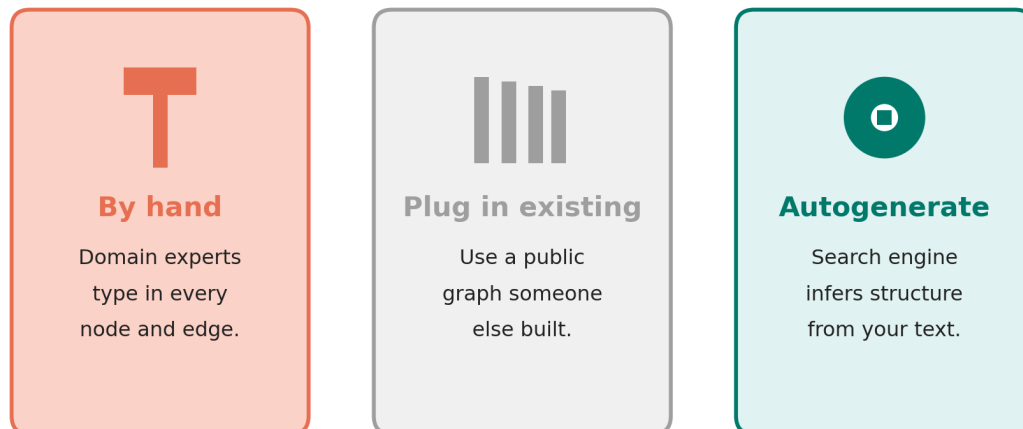
What other movies are in the Marvel Cinematic Universe?

Who else in our company has worked with this client?

A keyword search engine, by default, doesn't know that advil and motrin are related — they're just two different sequences of characters. A knowledge graph does know, because someone (or some automated process) drew a line between them.

Three ways to build a knowledge graph

If you want a knowledge graph for your own product, there are three main paths.



Option 1: By hand. You sit down with someone who knows the domain — a doctor, a lawyer, a librarian, an experienced product manager — and you type in every node and every edge. This is slow. It's expensive. It produces beautiful, precise graphs that nobody can argue with, and they're outdated the moment you stop maintaining them. Specialized industries that need very high accuracy (medical coding, legal taxonomies, drug interaction databases) do exactly this. It's the right tool when the cost of a wrong edge is high and the world doesn't change too fast.

Option 2: Plug in an existing one. Several large knowledge graphs already exist, built by other people. Wikipedia has a structured-fact cousin called Wikidata. There's ConceptNet, full of common-sense connections between everyday concepts. And, increasingly, large language models themselves act like implicit knowledge graphs — you can ask them, in plain English, what's related to what. The advantage is speed: you get a huge graph for free. The catch is that these graphs don't know about *your* domain. They won't know your product names, your customer segments, your internal jargon, the words your users actually type into your search bar.

Option 3: Have the search engine build one for you. Feed your own documents to a search engine and let it work out the relationships from the text itself. This is the option this chapter is about. The advantage: it knows about *your* terminology, it updates itself as new documents arrive, and it's free in the sense that you already have the documents and the search engine. The catch: it depends on having enough content. A graph built from three documents won't be very useful. A graph built from three million might be excellent.

REMEMBER

These three options aren't mutually exclusive. The most powerful real-world systems combine all three — autogenerate the bulk of the graph from content, plug in a public graph for common-sense facts, and sprinkle in hand-curated edges where accuracy really matters.

Where this chapter is heading

The rest of the chapter is about Option 3. Each section adds one piece:

- Section 2 shows that you don't actually need a separate graph database. The search engine you may already have is, in a real sense, a graph database in disguise.
- Section 3 shows two techniques for pulling structured facts out of plain text — useful if you want a clean, explicit knowledge graph as a side effect of your indexing.
- Section 4 shows a different, more powerful idea: leave the facts in the text and let the engine discover relationships on the fly, every time someone searches.
- Section 5 ties it together — how the pieces fit, and what they let you build that you couldn't before.

By the end you'll have a clear mental model of one of the most useful tricks in modern search.

2. Your Search Engine Already Knows More Than You Think

Here's the surprise this section is built around: the search engine you may already have is, in a meaningful sense, a knowledge graph database. You don't need to spin up Neo4j. You don't need to pay for a separate system. The graph is already there, hiding in plain sight inside the engine.

To see why, we need to look at what's actually inside a search engine. Most explanations skip this part. We won't.

What's inside a search engine

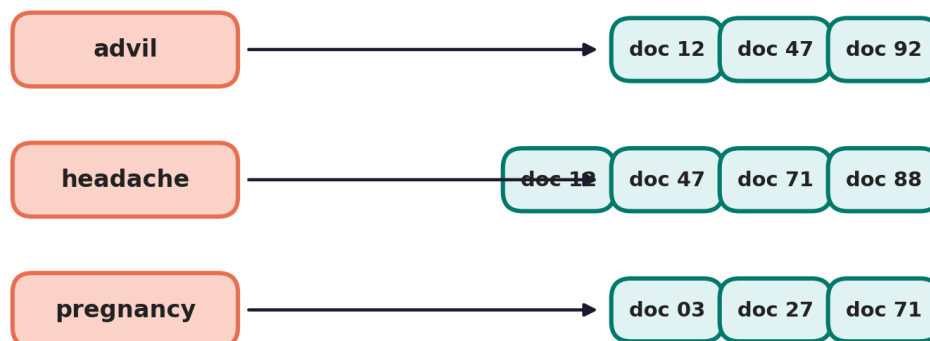
Imagine you have a library of ten thousand documents. Forum posts, product descriptions, support tickets — pick anything. To search them quickly, the engine builds two reference structures while it ingests the documents. Just two.

The inverted index

The first one is called the *inverted index*. It answers the question: *for each unique term that appears anywhere in my documents, which documents contain it?*

If your collection were a paper library, this is the index in the back of every book — but applied across the whole library. Look up *advil*, and you get a list of every document that mentions *advil*. Look up *headache*, and you get a list of every document that mentions *headache*.

Inverted index: term → documents



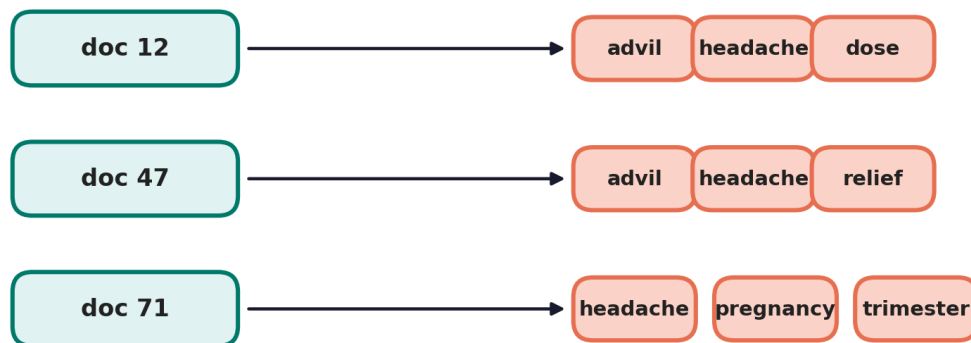
For each term, the engine knows which documents contain it.

The inverted index is what makes search fast. When you type *advil* in the search box, the engine doesn't read through ten thousand documents looking for the word. It looks the word up in the inverted index, gets back a list of document IDs, and returns those. Same idea as the index in the back of a book — flip to the page, don't read the whole book.

The forward index

The second structure goes the other direction. It's called the *forward index* — sometimes called the *uninverted index* in older texts, because it's the reverse of the inverted one. It answers: *for each document, which terms appear in it?*

Forward index: document → terms



For each document, the engine knows which terms appear in it.

If the inverted index is the index in the back of a book, the forward index is the table of contents — for each chapter, here are the words. Same information, organized the other way around.

Most search engineers know about the inverted index. The forward index gets less attention because most queries don't need it. But it turns out to be the secret to turning a search engine into a knowledge graph.

TECHNICAL

In Apache Solr and similar engines, the forward index is sometimes called "doc values" — values stored per document that you can quickly look up by document ID. The same information was historically used for sorting and faceting, where you need to know what values appear in matching documents. You can skip the terminology, but if you ever see "doc values" in a search engine's documentation, this is what they're talking about.

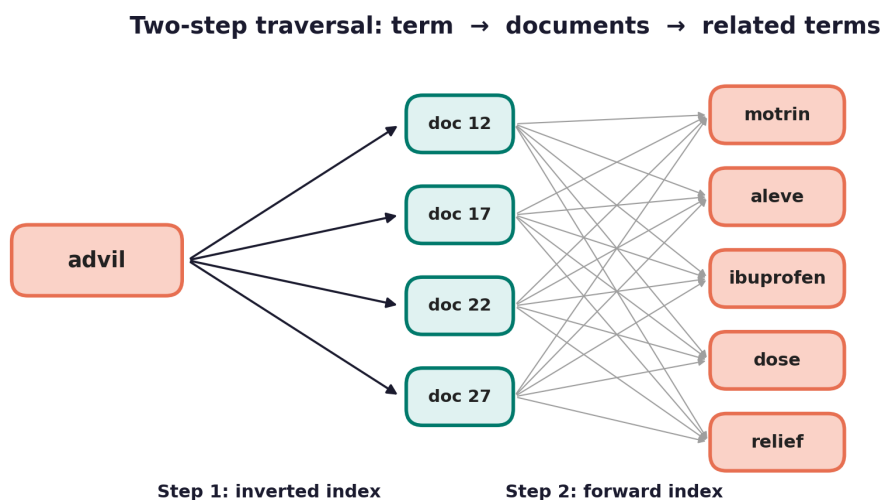
Two indexes, one graph

Here's the move that turns this into something graph-like.

If you have the inverted index, you can go *from a term to a set of documents*. You start with *advil*, and you get back doc 12, doc 47, doc 92.

If you have the forward index, you can go *from a document to a set of terms*. You start with doc 12, and you get back the terms it contains — *advil*, *headache*, *dose*.

Put those together and you can do a two-step walk. Start with a term. Go to the documents containing it. Then go to all the *other* terms in those documents.



That's a graph traversal. The starting term is one node. Each document is a node. Each other term is a node. The "appears in" relationship is an edge. You walked from one node, through some intermediate nodes, to a set of destination nodes.

We just walked a graph using nothing but a search engine.

REMEMBER

A graph traversal in this setup is two index lookups in disguise. Inverted index gets you from a term to documents. Forward index gets you from those documents to other terms. Composed, they let you walk through your content as if it were a graph.

This is the whole secret. Once you have both indexes — and any modern search engine does — you have the bones of a knowledge graph for free.

Why this matters

So what? Why is this worth a section?

Because building and maintaining a separate graph database is expensive. It's a second system to deploy, monitor, back up, secure, scale, and keep in sync with whatever else has your content. Lots of companies that started this journey by buying a dedicated graph database have eventually realized that for many use cases, the search engine they already had was enough.

In particular, *autogenerated* knowledge graphs — the kind we're building in this chapter — work especially well in a search engine. Every time you add a new document, the indexes automatically update. The graph updates with them. No separate ETL job. No nightly batch. No graph rebuild.

TIP

This doesn't mean a dedicated graph database is never the right tool. If your graph is mostly hand-curated, has rich and consistent edge types, and is queried in deeply nested ways (think six-step traversals through typed relationships), a real graph database may pay for itself. But if your graph is implicit in your text and your queries are one or two steps deep — the most common case — the search engine you have is plenty.

What's coming next

You now have a mental model of how the bones of a knowledge graph live inside a search engine. The next two sections show what you can do with that.

Section 3 takes the explicit route. We'll show two techniques for pulling clean, well-formed facts out of your text — sentences like "Lucy is friends with Charlie Brown" — and turning them into nodes and edges you can store.

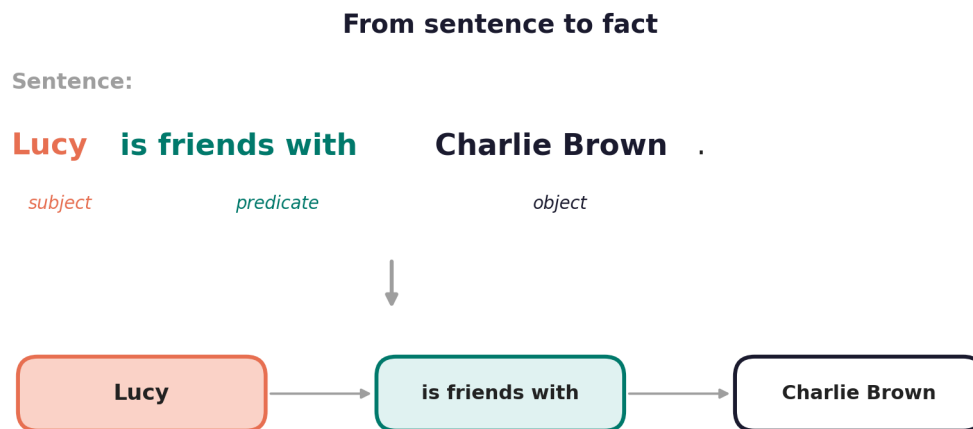
Section 4 takes a different route. Instead of extracting the facts in advance, we'll leave them in the text and let the engine discover relationships at query time, based on how words appear together. This turns out to be the more flexible, more powerful trick.

You'll see both, and you'll get a feel for when each one is the right choice.

3. Teaching a Computer to Read Between the Lines

Sometimes you don't want to *infer* relationships from your text. You want to *extract* them, cleanly and explicitly, and store them in your knowledge graph as crisp little facts.

A fact, in this world, is a tiny three-part thing. It has a *subject*, a *predicate* (the relationship), and an *object*.



"Lucy is friends with Charlie Brown" becomes the fact (Lucy, "is friends with", Charlie Brown). Three parts. Two nodes and the edge between them. People in the knowledge-graph world call this a *triple* — three parts, hence the name. You'll also see it called an *RDF triple*, where RDF stands for "Resource Description Framework," which is the name of an old web standard for writing facts like this down. Don't be intimidated by the acronym. A triple is just (*thing, relationship, thing*).

The question of this section is: how do you turn a sea of text into a pile of triples? Two main techniques, with very different trade-offs.

Technique 1: Read the grammar

The first technique is to teach the computer to read like a third-grader doing sentence diagrams.

Every English sentence has parts of speech. There's usually a subject (who or what the sentence is about), a verb (what they're doing), and often an object (what they're doing it to). A natural-language processing library, like spaCy, can parse a sentence and identify these parts automatically. Once you know which word is the subject and which is the object, you have your triple.

Here's the rough flow:

1. Take a sentence.
2. Have an NLP library label every word with its part of speech.
3. Identify the subject, verb, and object.
4. Output (subject, verb, object).

This works beautifully on simple sentences. *"Data scientists build machine learning models."* The subject is *data scientists*, the verb is *build*, the object is *machine learning models*. Triple: (data scientists, build, machine learning models). Done.

It works less well on messy sentences. Pronouns get in the way (*"They also write code"* — they who?). Complex sentences with multiple clauses produce noisy triples. Verbs come in many forms: *build*, *building*, *built*, *builds*, *will build* — the same relationship, five different surface words. Synonyms are even worse: *build*, *create*, *develop*, *construct*, *make*, *write* might all mean the same thing in context, but the extraction process treats them as separate edge types.

WARNING

When you extract facts this way, the cleanliness of the result depends heavily on how much you cleaned up the text first. Pronouns need to be resolved (replace *they* with what *they* refers to). Verbs need to be normalized (treat *build*, *built*, and *building* as the same edge). Even with all that, you'll get noise. This is fine if the next step is some kind of aggregation or fuzzy match. It's painful if you need the output to be database-clean.

Grammar-based extraction is flexible — it can pull out any kind of relationship the text describes — but it's noisy. Modern systems often use larger language models to do this more accurately. The trade-off is the same: more flexibility, more noise.

Technique 2: Look for specific patterns

The second technique sidesteps the noise problem by focusing on just one kind of relationship.

It turns out that one specific relationship is unusually well-behaved in English: the *is-a* relationship. Whenever you say one thing is a kind of another thing, English provides surprisingly few, surprisingly consistent ways to say it. A linguist named Marti Hearst noticed this in 1992 and wrote down the patterns.

Hearst patterns spot "is a" relationships

Pattern match:

Tools **such as** **hammers** and **screwdrivers** are widely used.

(hypernym) (trigger)

(hyponyms)



hammer is_a tool

screwdriver is_a tool

Here are some of those patterns, in plain English:

"Tools, such as hammers and screwdrivers, ..." → hammer is_a tool, screwdriver is_a tool.

"Vegetables including carrots, broccoli, and spinach" → carrot is_a vegetable, etc.

"Hammers and other tools" → hammer is_a tool.

"Carrots, broccoli, spinach, and other vegetables" → carrot is_a vegetable, etc.

"Phillips heads are a type of screwdriver" → phillips head is_a screwdriver.

There are about fifty of these patterns in total. Together they catch most of the ways English speakers express *is-a* relationships in writing.

Run those patterns over enough text and you get back a clean, fairly accurate, *is-a* graph. Sometimes called a *taxonomy*. You can use it to answer "what kind of X is this?" or "what are all the X's mentioned in our documents?" — exactly the kinds of questions a category-based search needs.

TIP

Hearst patterns work best on text where authors are actively trying to be informative — encyclopedias, forum posts, product descriptions. They work poorly on tweets, chat logs, and casual conversation, where people skip the explicit *is-a* phrasings ("I just got an iPhone" doesn't tell you an iPhone is a phone — but every product page on Apple's site does).

Hyponym and hypernym — two words you'll see

Two words you'll see in this corner of the world:

A **hyponym** is the more specific thing. *Hammer* is a hyponym of *tool*. *Carrot* is a hyponym of *vegetable*. *Advil* is a hyponym of *medication*.

A **hypernym** is the more general thing. *Tool* is a hypernym of *hammer*. *Vegetable* is a hypernym of *carrot*. *Medication* is a hypernym of *advil*.

(There's a third word, *meronym*, for part-of relationships, like "wheel is a meronym of car." That one's less common, and we won't use it.)

Hearst patterns are essentially a hyponym-extractor. They specifically find pairs of words in an is-a relationship and produce a cleaned-up list of them.

Two techniques, two trade-offs

Both techniques produce real facts. Both have their place.

The grammar-based technique gives you breadth: any kind of relationship you can name. The catch is noise — you'll spend real effort cleaning up the output before you can use it.

The pattern-based technique gives you cleanliness, but only for one relationship type. If you only ever needed is-a, you'd be perfectly happy. Most real systems need more.

Either way, both techniques have one big property in common: they *take the facts out of the text* and put them somewhere else, as discrete edges in a graph. That's useful, but it's lossy. You lose the surrounding context. You lose the nuance of *how* the document said something. And every fact in your graph is a fact you committed to in advance, whether or not anyone ever asked a question that needed it.

The next section shows a different, more flexible approach. Instead of extracting facts up front, we leave the facts in the text and let the search engine *discover* relationships on the fly, in response to whatever question someone asks. It turns out to be enormously powerful.

REMEMBER

Up-front extraction (this section) and on-the-fly discovery (next section) aren't competitors. They're complementary. Many real systems do both — extract the most important, stable facts as explicit edges, and let everything else fall out of relatedness scoring at query time.

4. Finding Hidden Connections in Your Words

This is the heart of the chapter. Brace yourself: by the end of it, you'll know a trick that makes a search engine feel meaningfully smarter, using nothing but the indexes you already have.

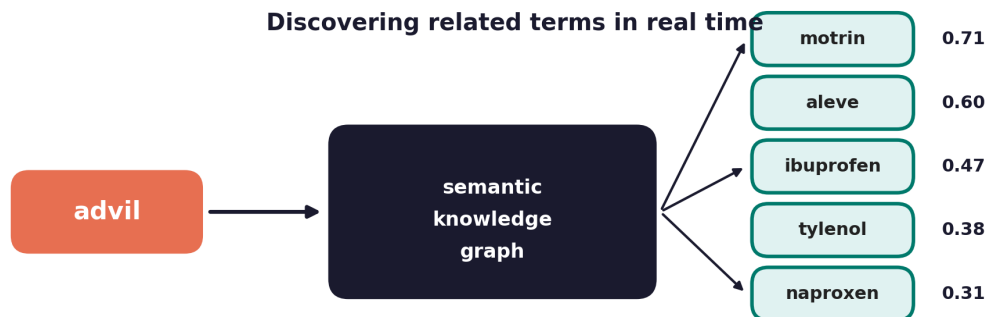
We'll start with an example, then explain how it works, then look at three things you can do with it.

The advil moment

Imagine you have a forum about health topics. Tens of thousands of posts where people ask each other about symptoms, medications, doctors, recoveries. A regular keyword search engine, fed your forum, can find documents containing the words people typed.

Someone types *advil* into the search box. Most engines, by default, return documents that contain the literal string *advil*.

But suppose instead of returning documents, we asked a different question: *what other words are conceptually similar to advil, in the world of this forum?*



Out come *motrin*, *aleve*, *ibuprofen*, *tylenol*, *naproxen*. Each one with a score.

Read those carefully. Motrin and ibuprofen are different brand names for the same drug as advil. Aleve and naproxen are different brand names for a closely related drug in the same class. Tylenol is a related but different painkiller. The engine ranked them roughly in order of how interchangeable they are.

Nobody told the engine that advil and motrin are related. There's no synonym file. There's no trained model. There's no embedding. The engine figured this out by looking at the documents in your

forum and asking itself a simple statistical question. The trick has a name — *semantic knowledge graph*, sometimes shortened to SKG — and we'll spend the rest of this section pulling it apart.

REMEMBER

A semantic knowledge graph isn't a thing stored on disk. It's a way of *traversing* the indexes you already have, at query time, to discover relationships that were never explicitly written down anywhere.

How does it know?

The idea is based on something linguists have known since the 1950s, called the *distributional hypothesis*. The hypothesis goes: *words that appear in similar contexts tend to have similar meanings*.

If you look at every health-forum document that contains the word *advil*, you'll see other words showing up alongside it: *headache, dose, pain, relief, cramp, fever*. And, importantly: *motrin, ibuprofen, aleve*. These last three appear in documents about exactly the same kinds of symptoms, the same kinds of dosages, the same kinds of side effects. They behave, statistically, like siblings of *advil*.

So one rough heuristic might be: *the words that appear most often in documents containing advil are most related to advil*.

That sounds reasonable. But it doesn't quite work. Here's why.

The most common words in any document about *advil* are also the most common words in every document. Words like *the, a, and, of, it, is*. These so-called *stopwords* would dominate any list of "words that appear most often in *advil* documents." They appear most often in *every kind of document*. They're not telling us anything specific about *advil*. They're telling us things about English.

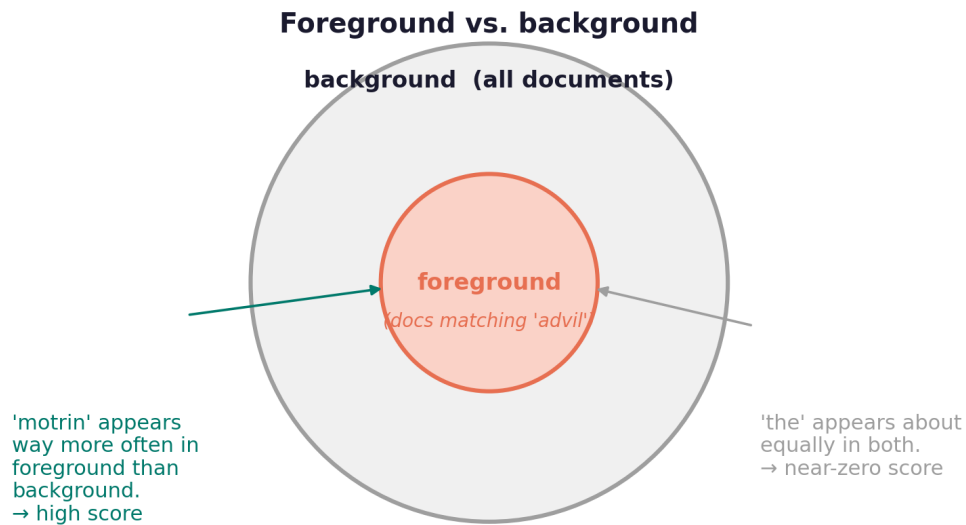
We need a smarter question.

The foreground / background trick

The smarter question goes like this: *Does this word appear in advil documents more often than it appears in documents in general?*

That's a comparison between two distributions:

- The **foreground** distribution: the documents that contain *advil*. We want to know how often each other word shows up in these specific documents.
- The **background** distribution: the entire corpus. How often does each word show up across *all* documents?

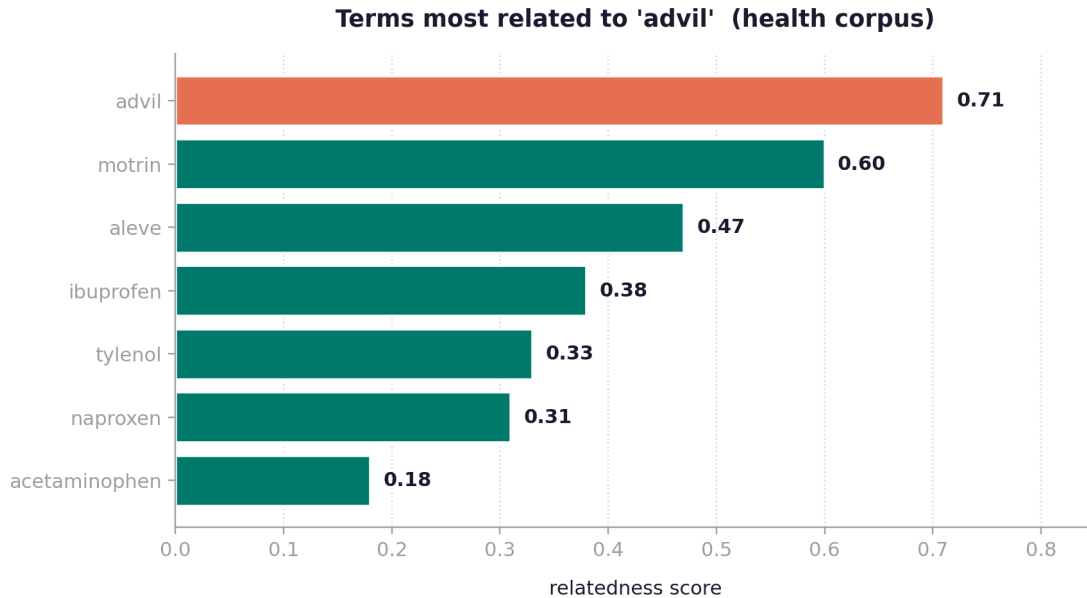


A word like *the* shows up in pretty much every document, so it's about as common in the foreground as in the background. Its over-representation in the foreground is zero. Boring.

A word like *motrin* shows up in lots of *advil* documents and almost nowhere else. Its over-representation in the foreground is huge. Interesting.

A word like *ibuprofen* is somewhere in between — extremely over-represented in *advil* documents, less than *motrin* but still strikingly so.

The engine assigns each candidate word a single number that captures this over-representation. That number is called the *relatedness score*. High score: the word is way more common in the foreground than you'd expect. Near-zero score: the word is about as common in both. Negative score: the word is *less* common in the foreground than in the background (think *recipe* showing up in *advil* documents — it doesn't, particularly).

**TECHNICAL**

The relatedness score is mathematically similar to a *z-score* — the number of standard deviations the observed count is from what you'd expect by chance. The score gets normalized so it lands in the range from -1 to 1 , where 1 means "perfectly related" and 0 means "no signal." You don't need to do the math yourself. You can think of it as a number that measures how surprised the engine should be to see this word in the foreground.

Notice what stopwords automatically get pushed to the bottom. *The* appears in 99% of documents, foreground or background. There's nothing surprising about seeing it in advil documents. The relatedness score is near zero. The trick of comparing the two distributions handles stopwords for free — you don't need a manual list.

How is this fast enough?

Reasonable question. To compute relatedness scores, the engine has to count, for every candidate word, how many times it appears in the foreground and the background. That sounds expensive.

It isn't, because — surprise — this is exactly what the inverted index already lets you do. For each candidate term, the engine looks up its document list in the inverted index, counts how many overlap with the foreground set, and computes the score. The forward index gives it the list of candidate terms (everything that appears in the foreground documents). Everything is one index lookup or one set intersection. The whole operation takes a few tens of milliseconds on a normal corpus, even at scale.

That's the deepest reason the trick lives inside the search engine. The same indexes that make keyword search fast are exactly the indexes you need to make semantic-relationship discovery fast.

What you can do with this

A relatedness query — give me a word, get back the related words — is the primitive. Three useful things you can build on top of it.

Use 1: Query expansion

Someone searches *advil*. Instead of finding only documents that literally contain the word *advil*, you expand the query to also include the most semantically related terms: *motrin*, *ibuprofen*, *aleve*. You weight each by its relatedness score, so documents about *advil* and *motrin* still rank above documents that mention only *aleve*.

The user's search results get dramatically better. They find documents that are *about the same thing*, even when those documents happened to use a different word.

Query expansion is the single most common use of semantic knowledge graphs. It often produces measurable, dramatic improvements in click-through rate and relevance — for free, without changing the underlying engine, without retraining anything.

TIP

You don't have to expand every query. The best results often come from expanding only the queries the engine isn't doing well on already — typically the niche, low-volume, technical terms where the corpus has plenty of related vocabulary that the user didn't think to type.

Use 2: Content-based recommendations

Take a document. Score every word in it by how related it is to a topic. The highest-scoring words become a kind of "fingerprint" of the document — *this document is about luke, force, darth vader, princess leia*. Use that fingerprint to find other documents whose fingerprint overlaps. You just built a "more like this" recommender, with zero training data, using the same indexes.

This is especially powerful in the "cold start" case — when you have new content and no user interactions yet. Behavior-based recommenders need someone to click on something first. Content-based recommenders work the moment the content is indexed.

Use 3: Typed relationships

The most flexible use, and the one that's hardest to summarize. So far we've talked about an unlabeled relationship — *related to*. But the same primitive can be steered to discover specific *kinds* of relationships.

Suppose you want to ask: *who is in love with Jean Grey?*

You can structure that as a traversal through a small graph: *"Jean Grey" → "in love with" → the answers*.

The engine treats the phrase *in love with* as just another node — a piece of context to narrow the foreground. The candidates that pop out the other side are exactly the names that co-occur strongly with both Jean Grey *and* the phrase *in love with*. Run that against the X-Men comics, and out come Wolverine, Cyclops, Phoenix.

The same primitive — comparing foreground to background — but now the foreground is the intersection of two conditions, not one. That's enough to discover typed edges that you never had to explicitly model.

This is the most expressive use of a semantic knowledge graph, and the one that takes the longest to internalize. It's the closest thing to "asking the corpus a structured question" — and it's all just index lookups under the hood.

A summary in one paragraph

A semantic knowledge graph isn't a data structure; it's a *way of asking the indexes a question*. You give the engine a starting point — usually a word or phrase. The engine finds all the documents that match that starting point, and then asks: *what other words appear in these documents much more than they appear elsewhere?* The answer is a ranked list of conceptually related terms. You can do this in real time, you can chain it together, you can steer it with extra context. The trick that makes the whole thing work is the foreground / background comparison: looking not for what's common, but for what's *over-represented*.

That single idea — over-representation — powers the magic in this chapter.

5. Putting It All Together: Smarter Search

Let's recap the journey. Five short paragraphs, one per section.

In Section 1 we said a knowledge graph is dots connected by lines, with real-world things and real-world relationships. We saw three ways to build one — by hand, by plugging in an existing one, or by autogenerating from your own content — and we said the rest of the chapter would be about that third option.

In Section 2 we said you don't need a separate graph database to build a graph. Every search engine already maintains two indexes: the inverted index (term → documents) and the forward index (document → terms). Composing those two lookups gives you a two-step graph traversal for free.

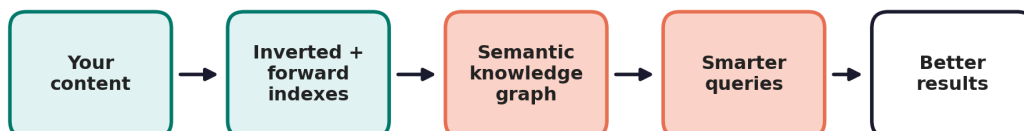
In Section 3 we looked at two ways to *extract* facts from text into an explicit knowledge graph. The grammar-based approach reads sentences and pulls out subject-predicate-object triples. The pattern-based approach (Hearst patterns) catches a specific kind of relationship — *is-a* — with high accuracy. Both work; both have trade-offs; both produce explicit graphs that live next to your content.

In Section 4 we saw a different idea: don't extract the facts in advance. Leave them in the text and let the engine *discover* relationships on the fly by comparing two distributions — the foreground (documents that match a query) and the background (the whole corpus). Words that are over-represented in the foreground are conceptually related to the query. The *advil* example: type *advil*, get back *motrin*, *aleve*, *ibuprofen*, no curation required.

And in this section, we'll pull all of that together into one picture and talk about what it lets you build.

The whole chapter, on one page

The chapter, on one page



Each arrow is something Chapter 5 walks you through.

Start at the left. You have content — documents, posts, descriptions, whatever. You index it into a search engine that maintains both the inverted and forward indexes. That gives you a semantic knowledge graph for free, traversable at query time. With the graph in hand, you can rewrite queries to be smarter — expanding them with semantically related terms — and the resulting search results are dramatically better than what plain keyword matching would have produced.

Five boxes, four arrows. Each arrow is something this chapter walked you through.

The picture is intentionally simple. Real systems add steps — there's usually a query parser, a re-ranker, possibly a machine-learned scoring model, possibly a personalization layer. But you can hold the whole shape of Chapter 5 in your head without those.

What this lets you do that you couldn't before

Three concrete capabilities.

Find documents that don't contain the words people typed. Plain keyword search can't find a document about *motrin* when someone types *advil*, no matter how relevant it is. A semantic-knowledge-graph-expanded search can. This is the most visible improvement and the easiest to demonstrate. It's also the one most directly tied to user satisfaction — people don't know your exact vocabulary, and they shouldn't have to.

Build "more like this" recommendations from text alone. No need to wait for users to click on things, no need to train an embedding model, no need to stand up a recommender service. The same indexes that power search also power recommendations, using the same primitives.

Ask the corpus structured questions. Beyond simple "related to" queries, you can use intermediate nodes — like the *in love with* example — to discover typed relationships that you never had to explicitly model. This is the most powerful and most under-used capability.

What the book covers next

This chapter introduced the engine — both the semantic-knowledge-graph idea and the extraction techniques that complement it. The next chapters of *AI-Powered Search* build on that engine.

Chapter 6 talks about *learning domain-specific language* from user signals — what queries do people actually type, what do they correct themselves to, what alternate spellings show up, what phrases are people treating as units? Some of those answers come from analyzing the same indexes you've been seeing here, just from a different angle.

Chapter 7 puts the pieces together into a full query interpretation pipeline. A user types a query; the system parses it, expands it with related terms (using the semantic knowledge graph from this chapter), rewrites it into a form the engine can run, and returns results that are notably better than what raw keyword matching would have produced.

Chapters 8 through 12 cover *reflected intelligence* — using user signals (clicks, dwell time, search reformulations) to keep improving the ranking over time. The semantic knowledge graph plays a supporting role there too.

But the foundational trick — turn your search engine into a knowledge graph by exploiting its two indexes — is what you came for, and you have it.

Three things to take with you

1. Use what you have. The search engine you may already have can do far more than match keywords. It's already a graph. Treat it as one.

1. The corpus is the source of truth. Semantically related terms don't have to come from a curated synonyms file. They come from how your content uses language. If you ever feel tempted to hand-build a synonyms list, ask first whether the corpus could just tell you.

1. Relatedness is just statistics. Don't be intimidated by the math. The whole trick is a comparison: how often does this word appear in the foreground compared to how often it appears in the background? That's it. Surprise is the signal.

You now have a complete mental model of Chapter 5. The next time someone says "we should add a knowledge graph to our search," you'll know what they mean — and you'll know whether you already have one.

Glossary

The terms we introduced, in one place, in plain English.

Background. In a relatedness query, the wider reference set — usually every document in your corpus. The yardstick we measure the foreground against.

Distributional hypothesis. The idea, going back to the 1950s, that words with similar meanings tend to appear in similar contexts. The intellectual foundation of the semantic knowledge graph.

Edge. A line in a graph. Connects two nodes. Usually has a label that describes the relationship. (*Lucy, is friends with, Charlie Brown*) uses *is friends with* as the edge.

Foreground. In a relatedness query, the smaller, query-specific set — the documents that match the starting query. We're interested in what's over-represented in the foreground compared to the background.

Forward index. A data structure inside a search engine that maps each document to the list of terms it contains. Sometimes called the *uninverted index* or *doc values*. Used here to walk from documents back to terms.

Hearst pattern. A linguistic template that reliably signals an *is-a* relationship in English text. Named after Marti Hearst, who described them in 1992. Examples: "*X such as Y and Z*", "*X including Y and Z*", "*Y and other X*".

Hypernym. The more general thing in an *is-a* relationship. *Tool* is the hypernym of *hammer*.

Hyponym. The more specific thing in an *is-a* relationship. *Hammer* is a hyponym of *tool*.

Inverted index. A data structure inside a search engine that maps each unique term to the list of documents containing it. The reason keyword search is fast.

Knowledge graph. A graph whose nodes are real-world things and whose edges are meaningful relationships. Used to answer questions keyword search can't.

Node. A dot in a graph. Stands for a *thing*. Also called a *vertex*.

Ontology. A formal description of which kinds of nodes and edges are allowed in a domain. Think *schema* for knowledge.

Query expansion. Taking a user's query and adding semantically related terms to it before searching. The most common use of a semantic knowledge graph.

RDF triple. A fact written as (*subject, predicate, object*). The plumbing of most explicit knowledge-graph formats. RDF stands for *Resource Description Framework*.

Relatedness score. A number that measures how over-represented a term is in the foreground compared to the background. High score: the term is conceptually related to the query. Near-zero: it's not.

Semantic knowledge graph (SKG). A way of traversing a search engine's indexes — at query time, with no precomputed graph — to discover related terms. The central trick of Chapter 5.

Taxonomy. A tree-shaped knowledge structure, usually built from *is-a* relationships. A simple but useful kind of knowledge graph.

Triple. Short for RDF triple. A three-part fact: (*subject, predicate, object*).

Vertex. Same as node. Mathematicians use *vertex*; everyone else says *node*.

Cheat sheet

If you remember nothing else from this chapter.

A graph is dots and lines. Nodes are things, edges are relationships. That's the whole vocabulary.

A search engine is secretly a graph. Its inverted index gives you term → documents. Its forward index gives you document → terms. Compose them and you can walk through your content as if it were a graph.

You can extract facts from text two ways. Grammar-based extraction (sentence → triple) is flexible but noisy. Hearst patterns (looking for "X such as Y") are accurate but only catch *is-a* relationships.

You can also discover relationships on the fly. Compare two distributions — foreground (docs matching a query) versus background (the whole corpus). Words over-represented in the foreground are semantically related. This is the semantic knowledge graph.

Three things to do with related terms. Expand queries to find documents that don't contain the exact words. Build content-based "more like this" recommendations. Use intermediate nodes to discover typed relationships you never modeled.

Surprise is the signal. Relatedness is a statistical measure of how surprised the engine should be to see a word in the foreground. Stopwords are unsurprising and get scored near zero. Real semantic siblings are surprising and get high scores. No synonym file required.